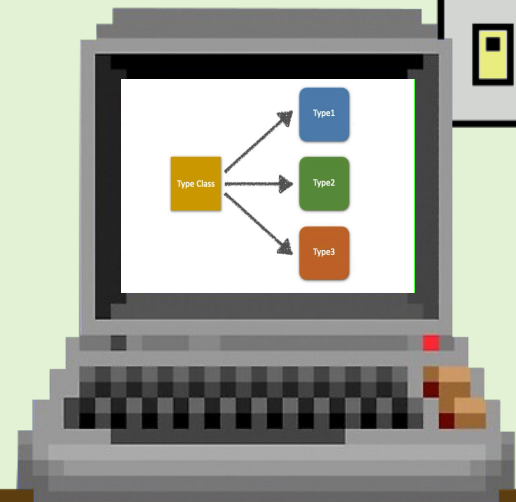


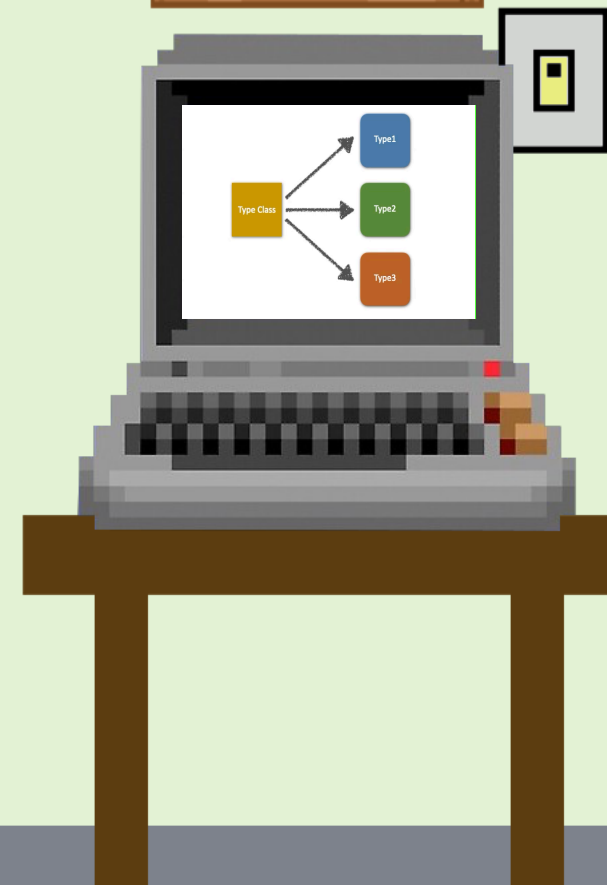
Classes de Tipos

Programação Funcional



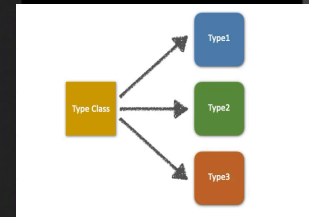
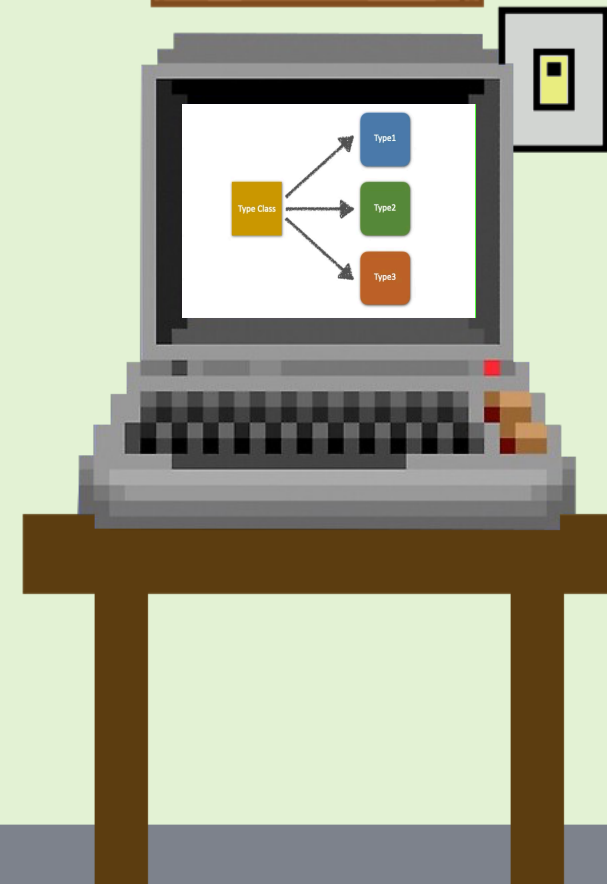
Classes de Tipos

- Muitas funções atuam sobre tipos diferentes
 - Ex: `length`, `filter`, `map`, ... (Polimorfismo paramétrico)
- Outras funções exigem uma nova implementação
 - Ex: operadores `+`, `-` e `*` (Sobrecarga)
- Coleção de tipos para os quais é definido um conjunto de funções que podem ter diferentes implementações
- Ex1: A classe `Eq` é formada por todos os tipos que podem ser verificados se são iguais ou diferentes
- Ex2: A classe `Num` é formada pelos tipos numéricos e sobrecarrega algumas operações aritméticas básicas
 - Os tipos `Int` e `Double` são instâncias da classe `Num`



Classes de Tipos

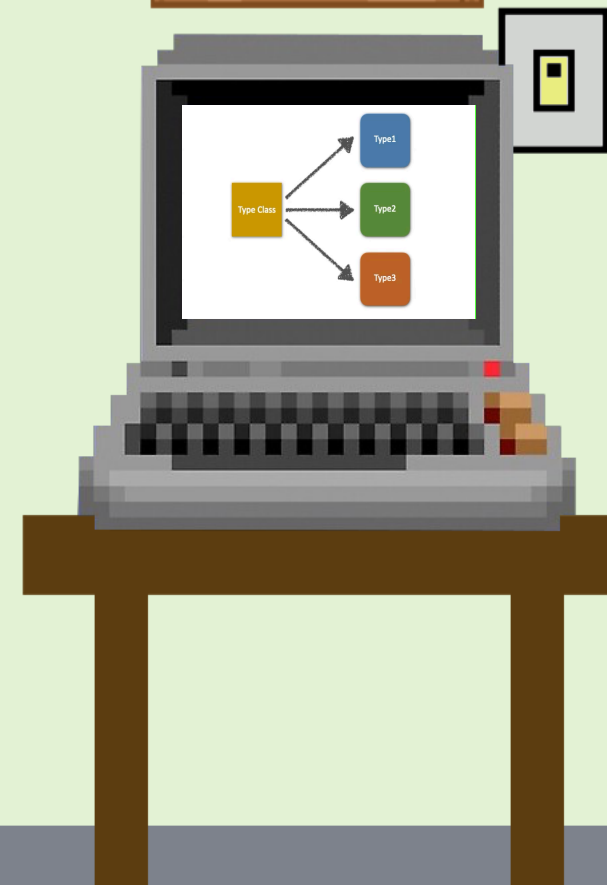
- Os tipos sobre os quais uma função sobrecarregada pode atuar formam uma coleção de tipos (type class)
- Ex1: A função abs é do tipo $\text{Num } a \Rightarrow a \rightarrow a$, ou seja, abs opera sobre argumentos de um tipo a, sendo a qualquer tipo que instancia a classe Num
- Ex2: $\text{todosIguais } m \ n \ p = (m==n) \ \&\& \ (n==p)$
 - Qual o tipo de todosIguais?
 - $\text{todosIguais} :: \text{Eq } a \Rightarrow a \rightarrow a \rightarrow a \rightarrow \text{Bool}$



Classes de Tipos

- Definem um conjunto de funções que podem ter diferentes implementações de acordo com cada tipo
- Exemplo:
 - Define a classe de tipo Equals como um tipo que implementa a função isEqual
 - isEqual é uma função que recebe dois argumentos que são instâncias de a e retorna um booleano

```
class Equals a where  
  isEqual :: a -> a -> Bool
```



Classes Pré-definidas

- Classe Eq

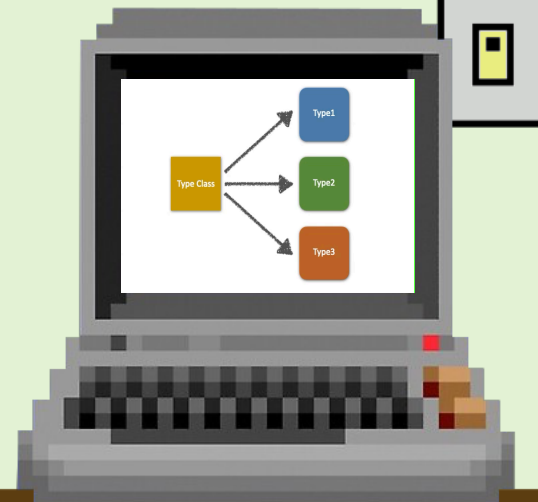
```
class Eq t where  
  (==), (/=) :: t -> t -> Bool
```

- Classe Ord

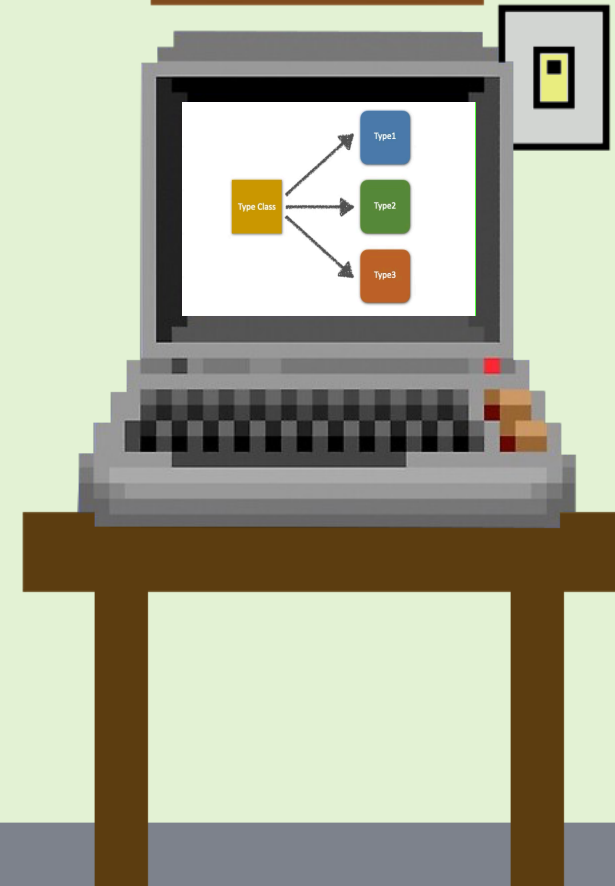
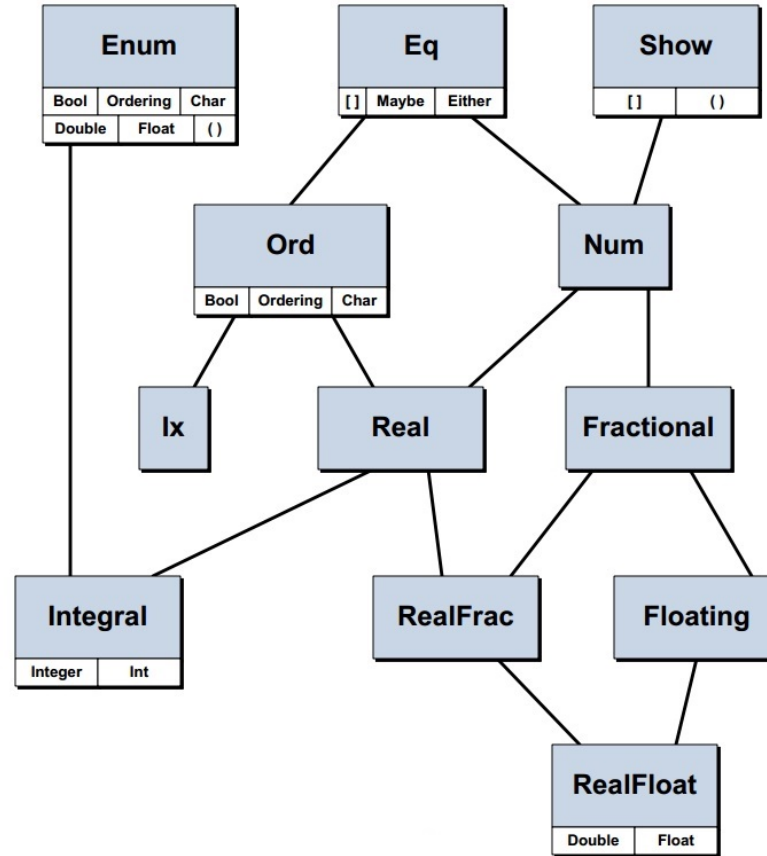
```
class Eq t => Ord t where  
  (<), (<=), (>), (>=) :: t -> t -> Bool  
  max, min :: t -> t -> t
```

- Classe Show

```
class Show a where  
  show :: a -> String
```



Hierarquia de Classes - Haskell



Exemplo

- Definição de função polimórfica com classe de tipo

```
todosIguais :: (Eq t) => t->t->t->Bool  
todosIguais m n p = (m==n) && (n==p)
```



Exemplo

- Implementação de classe de tipo

```
class Located a where  
  getLocation :: a -> (Int, Int)
```

```
data Point = Point { coordX::Int, coordY::Int}
```

```
instance Located Point where  
  getLocation p = (coordX p, coordY p)
```

```
origem:: (Located t)=> t -> Bool  
origem p = getLocation p == (0,0)
```

